UNIVERSIDAD DON BOSCO
ESCUELA DE COMPUTACIÓN
PROYECTO FASE II

Propuesta de despliegue en Microsoft Azure

1. Descripción del proyecto

SmartStock AI es una aplicación web para apoyar el control de inventario de pequeños negocios. En el proyecto se encontraron ASP.NET Core con C#, Entity Framework Core y SQL Server. El código está separado en proyectos de dominio, aplicación, infraestructura, web y pruebas.

Entre las funciones del sistema están los productos, las categorías, los proveedores, las compras, las ventas, los movimientos de inventario, el tablero principal y los reportes. También hay código para generar recomendaciones de inventario con inteligencia artificial.

2. Objetivo de la propuesta

El objetivo es indicar cómo podría publicarse SmartStock AI en Microsoft Azure sin tener que administrar un servidor físico o una máquina virtual. La propuesta separa la aplicación, la base de datos y el servicio de IA para que se puedan configurar y ampliar por separado.

La consigna de la Fase II pide seleccionar los servicios de Azure, explicar sus beneficios, presentar costos aproximados y mostrar la arquitectura de despliegue. No es obligatorio desplegar el sistema realmente; sí es necesario documentar una forma posible de hacerlo.

3. Servicios de Azure seleccionados

Azure App Service para Linux

Se propone usar App Service para ejecutar la aplicación ASP.NET Core desde una imagen Docker. Azure administra el entorno donde corre la aplicación y permite configurar el sitio, variables de entorno, HTTPS y el tamaño del plan. Para una primera prueba se considera el plan Basic B1, que tiene 1 vCPU y 1.75 GB de memoria según la página de precios de Microsoft

Azure Container Registry

Azure Container Registry (ACR) guardará la imagen Docker de SmartStock AI. App Service podrá descargar desde allí la versión que se quiera ejecutar. Para una prueba académica se propone empezar con el nivel Basic y conservar pocas versiones de la imagen

Azure SQL Database

La base de datos se alojaría en Azure SQL Database. Entity Framework Core se conectará a ella mediante una cadena de conexión configurada en Azure, no escrita directamente en el código. Para una prueba pequeña se puede evaluar el nivel Basic; tiene límites de rendimiento y hasta 2 GB de almacenamiento, por lo que debe revisarse si alcanza para los datos y las pruebas

Azure OpenAI

Para las recomendaciones se propone Azure OpenAI, siempre que la suscripción y la región elegida permitan usar un modelo adecuado. Esto no está listo solo por crear el recurso: el código de SmartStock AI debe configurarse para el endpoint y el modelo desplegado, y la integración debe probarse antes de decir que funciona.

Azure Key Vault y Application Insights

Key Vault se usaría para guardar valores sensibles, como claves y contraseñas. Application Insights y Azure Monitor ayudarían a revisar errores, tiempos de respuesta y problemas de conexión. El costo de monitorización depende de la cantidad de datos que se registren

4. Arquitectura de despliegue propuesta

El funcionamiento propuesto sería el siguiente:

Usuario desde el navegador

| conexión HTTPS

Azure App Service para Linux (aplicación ASP.NET Core dentro de Docker)

| Entity Framework Core y conexión cifrada

Azure SQL Database (datos de productos, ventas, compras e inventario)

| solo cuando se solicita un análisis

Azure OpenAI (generación de recomendaciones de inventario)

Azure Container Registry guarda la imagen Docker que usa App Service. Key Vault protege los secretos y Application Insights registra información útil para detectar errores. Estos servicios se conectan a la aplicación, pero cumplen funciones distintas.

Flujo de uso

1. El usuario abre la dirección web mediante HTTPS.

2. App Service ejecuta la aplicación SmartStock AI desde la imagen Docker publicada en ACR.

3. La aplicación consulta y actualiza Azure SQL Database mediante Entity Framework Core.

4. Cuando el usuario solicita recomendaciones, la aplicación prepara las métricas necesarias y llama al servicio de IA. No se deberían enviar contraseñas ni datos que no hagan falta para el análisis.

5. Los errores y tiempos de respuesta se revisan mediante los registros de la aplicación y Application Insights.

5. Beneficios de la propuesta

- Menos administración del servidor: App Service se encarga de una parte del mantenimiento de la plataforma web.
- Uso de Docker: se publica una imagen concreta de la aplicación, lo que ayuda a repetir la instalación en otro entorno.
- Base de datos administrada: Azure SQL Database incluye herramientas de respaldo y permite cambiar el nivel de servicio si la carga crece.
- Separación de responsabilidades: la aplicación, la base de datos y la IA se pueden mantener y ajustar por separado.
- Seguridad: los secretos pueden mantenerse fuera del repositorio y el sitio puede exponerse mediante HTTPS.
- Monitoreo: los registros permiten revisar fallos de la aplicación, la base de datos o el servicio de IA.

6. Costos aproximados

La siguiente tabla es un presupuesto de planificación para una aplicación de baja carga, una base de datos pequeña y pocas solicitudes de IA. Los importes están en dólares estadounidenses por mes. No son una cotización final: los precios dependen de la región, la configuración y el consumo. Antes de crear los recursos se debe introducir la configuración exacta en la Calculadora de precios de Azure

Servicio	Supuesto	Costo mensual aproximado
App Service Linux, Basic B1	Una instancia para una aplicación con poca carga. La página de precios consultada muestra una referencia de $13.14/mes; confirmar región.	$13-$15
Azure Container Registry, Basic	Un registro y pocas imágenes/versiones. Costos adicionales si se supera el almacenamiento incluido.	$5
Azure SQL Database, Basic	Base de prueba pequeña; 5 DTU y hasta 2 GB. Confirmar el precio con región y configuración.	$5-$15
Key Vault y Application Insights	Pocas operaciones y poco volumen de registros.	$0-$5
Azure OpenAI	Uso ocasional de recomendaciones; depende del modelo y los tokens procesados.	$1-$10
Total de referencia	Un entorno, carga baja y sin recursos de alta disponibilidad.	Aproximadamente $25-$50/mes

El total se debe tomar como una estimación inicial, no como un precio garantizado. No incluye impuestos, dominio propio, tráfico alto, entornos adicionales, conectividad privada ni alta disponibilidad. Las llamadas de IA y los registros pueden aumentar el consumo según el uso.

7. Configuración necesaria

En App Service se tendrían que configurar los valores de producción y los datos para conectarse a la base de datos. Las claves y contraseñas reales no deben escribirse en el repositorio ni mostrarse en capturas.

- ASPNETCORE_ENVIRONMENT=Production
- ConnectionStrings__DefaultConnection: cadena de conexión real de Azure SQL Database, con cifrado activado.
- AI__Provider: proveedor de IA configurado en el código.
- AI__Endpoint: endpoint real del servicio de IA, si se configura Azure OpenAI.
- AI__Model: nombre del despliegue del modelo disponible en Azure.
- AI__ApiVersion y AI__ApiKey: versión de API compatible y clave real, guardada de forma segura.
- WEBSITES_PORT=8080: configurar este puerto si la imagen sigue escuchando en 8080; luego verificar el arranque

.

8. Pasos de implementación

1. Corregir el Dockerfile para que las rutas coincidan con las carpetas reales del repositorio.

2. Revisar las variables que usa docker-compose.yml y compararlas con .env.example. Crear un archivo .env local, completar los valores necesarios y no subirlo a Git.

3. Corregir la configuración de la aplicación y el registro de los servicios de IA en Program.cs. Después ejecutar restore, build y las pruebas automatizadas.

4. Crear un grupo de recursos en Azure y elegir una región compatible con los servicios que se van a usar.

5. Crear Azure SQL Database y preparar la cadena de conexión segura. Verificar las migraciones y que la aplicación pueda conectarse a la base.

6. Crear Azure Container Registry, construir la imagen Docker y publicarla con una etiqueta de versión.

7. Crear App Service para Linux y configurarlo para descargar la imagen de ACR. Revisar el puerto de escucha y los registros del contenedor.

8. Si se usará Azure OpenAI, crear el recurso y el despliegue de modelo, configurar los valores reales y probar el flujo de recomendaciones.

9. Configurar Key Vault y Application Insights si se usarán en el despliegue.

10. Probar el acceso al sistema, inicio de sesión, productos, compras, ventas, inventario, reportes, base de datos y recomendaciones. Guardar evidencia de las pruebas sin mostrar contraseñas ni claves.

9. Seguridad y mantenimiento

El sitio debe usar HTTPS. La conexión a Azure SQL Database debe tener cifrado y validar el certificado. Las reglas de acceso de la base de datos no deben permitir conexiones abiertas para cualquier origen si no son necesarias.

Antes de un despliegue real hay que cambiar o deshabilitar las credenciales de demostración que vienen en el código. También se recomienda configurar una política de respaldos, probar al menos una restauración y limitar el volumen de registros para controlar su costo.

El nivel inicial de App Service y la base Basic son solo para una prueba pequeña. Si aumenta la cantidad de usuarios o de operaciones, se deben revisar los tiempos de respuesta, memoria, CPU y carga de la base de datos antes de subir el nivel de servicio.

10. Conclusión

Para SmartStock AI se propone usar App Service para la aplicación, ACR para almacenar la imagen Docker y Azure SQL Database para los datos. Azure OpenAI puede usarse para las recomendaciones si se termina la integración y se dispone de un despliegue de modelo. Key Vault y Application Insights completarían la parte de seguridad y monitoreo.

El costo de referencia es de $25 a $50 al mes para un uso pequeño. Antes de contratar los servicios hay que revisar la calculadora de Azure. También es necesario corregir las rutas de Docker, unificar las variables y completar el registro de los servicios de IA antes de considerar que el sistema está listo para desplegarse.

11. Fuentes consultadas

[1] Microsoft Azure. App Service para Linux, precios: https://azure.microsoft.com/en-us/pricing/details/app-service/linux/

[2] Microsoft Azure. Azure Container Registry, precios: https://azure.microsoft.com/en-us/pricing/details/container-registry/

[3] Microsoft Azure. Azure SQL Database, precios: https://azure.microsoft.com/en-us/pricing/details/azure-sql-database/

[4] Microsoft Azure. Key Vault, precios: https://azure.microsoft.com/en-us/pricing/details/key-vault/

[5] Microsoft Azure. Azure Monitor, precios: https://azure.microsoft.com/en-us/pricing/details/monitor/

[6] Microsoft Azure. Calculadora de precios: https://azure.microsoft.com/en-us/pricing/calculator/

[7] Microsoft Learn. Configurar un contenedor personalizado en Azure App Service: https://learn.microsoft.com/en-us/azure/app-service/configure-custom-container

[8] Microsoft Learn. Planificar y administrar costos de Azure SQL Database: https://learn.microsoft.com/en-us/azure/azure-sql/database/cost-management?view=azuresql